

Chapter 11: Delivering Letters

Street-Based Routing with Evolutionary Algorithms
Nature Inspired Optimisation for Delivery Problems (2022)

N. Urquhart

Lecture Notes — Self-Contained Reader Edition

Table of Contents I

- 1 11.1 Introduction
- 2 11.2 Street Based Routing
- 3 11.3 Implementation
- 4 11.4 Comparison with TSP
- 5 11.5 Conclusions

11.1 Introduction – The Postal Delivery Problem I

Postal workers, charity collectors, census takers, and political canvassers all share a common challenge: they must visit every house in a given area while walking as short a total distance as possible. This is an example of a **house-to-house delivery** routing problem in an urban setting.

Problem Statement

Given a set of delivery addresses distributed across a network of streets, find the shortest walking route that visits every address exactly once, starting and ending at a fixed depot (e.g. a postal sorting office or a parked van).

Why not just use the Travelling Salesperson Problem (TSP)? The natural first instinct is to model each delivery address as a city in a TSP. For n addresses, the TSP has a solution space of $n!/2$ possible routes (dividing by 2 because the route can be traversed in either direction, giving the same total distance). For just 50 addresses this is already $50!/2 \approx 1.5 \times 10^{63}$, a number far beyond exhaustive search. For hundreds of addresses — which is typical for a single postal walk — the TSP becomes computationally intractable.

11.1 Introduction – The Postal Delivery Problem II

Key Challenge

The TSP scales poorly because it treats every single address as an independent entity. It ignores the fact that in urban environments, addresses cluster naturally into streets, and streets connect at junctions. This structure can be exploited to reduce the search space dramatically.

11.1 Introduction – The Postal Delivery Problem III

Urban structure offers an opportunity. Within a city, individual addresses group together in a very regular way:

- Addresses sit on one or both sides of a *street section* — the segment of a street between two consecutive junctions.
- Street sections connect to each other at *junctions* (corners, T-junctions, roundabouts, etc.).
- A delivery worker walking down a street section will naturally deliver to all addresses on that section before moving to the next.

The figure below shows an extract from a real city map where addresses have been grouped into street sections. Red squares mark individual delivery addresses; orange circles mark junctions.

11.1 Introduction – The Postal Delivery Problem IV

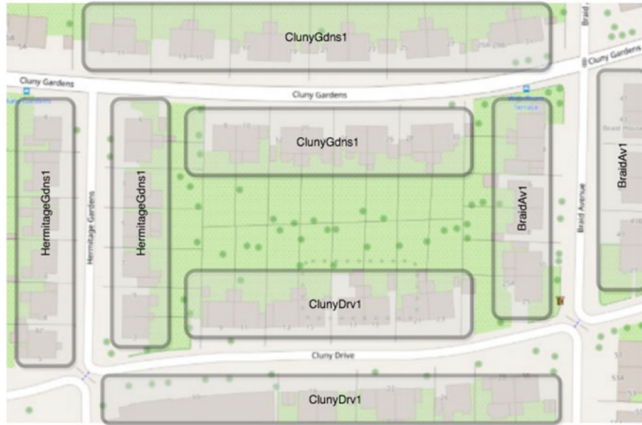


Fig. 11.1 Four urban streets, with the houses grouped by street section and side. *Base map and data from OpenStreetMap and OpenStreetMap Foundation (<https://www.openstreetmap.org/copyright>)*

- All of the buildings are on the same side of the road.
- All of the buildings are located between two junctions.

11.2 Street Based Routing – Representation I

The central idea of SBR is a **chromosome** (the EA's encoding of a candidate solution) whose genes are *street sections* rather than individual addresses. Each gene is simply the name (or identifier) of a street section.

Definition: SBR Chromosome

An SBR chromosome is an ordered list of street-section identifiers. The order of genes in the chromosome determines the order in which the delivery worker visits the street sections. Each gene is expanded into its full list of individual delivery addresses during *decoding*.

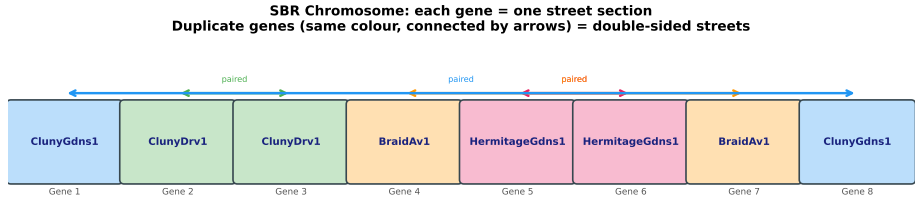
A worked example illustrates this well. Suppose the delivery area contains five street sections: ClunyGdns1, ClunyDrv1, BraidAv1, HermitageGdns1, and their double-sided counterparts. A valid chromosome (as shown in Table 11.1 of the book) is:

ClunyGdns1	ClunyDrv1	ClunyDrv1	BraidAv1	HermGdns1	HermGdns1	BraidAv1	Cluny
Gene 1	Gene 2	Gene 3	Gene 4	Gene 5	Gene 6	Gene 7	Gen

11.2 Street Based Routing – Representation II

Notice that ClunyDrv1 and HermitageGdns1 each appear *twice* in the chromosome. This is because these sections have houses on **both sides** of the road. When a section appears twice, the decoder treats the two appearances as two separate operations: one for each side of the street. When the two appearances are *adjacent* in the chromosome, the decoder delivers to both sides in a single pass (a U-turn at one end). When they are *separated*, the decoder delivers to each side on two separate visits.

11.2 Street Based Routing – Representation III



The figure above shows the chromosome as a sequence of colour-coded boxes. Same-coloured pairs (connected by arrows) are the two genes for a double-sided street section. Same colour means same street section; different positions in the sequence mean different visits.

11.2 Street Based Routing – Representation IV

Why Pairs for Double-Sided Streets?

A double-sided street section has two distinct sets of addresses — one on the left footpath and one on the right. The delivery worker can choose to deliver both sides in one sweep (U-turn pattern) or visit them on separate occasions. The SBR chromosome captures this naturally: two identical consecutive genes = one combined sweep; two identical non-adjacent genes = two separate visits.

11.2 Street Based Routing – Representation V

Three things the decoder must determine for each section:

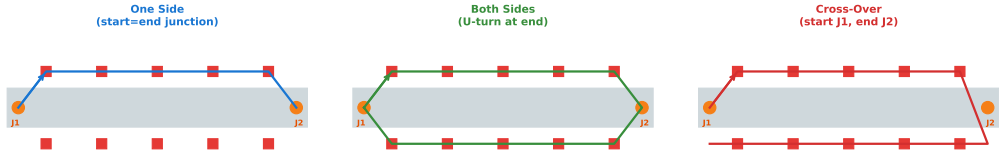
- ❶ **The starting junction:** which end of the section to enter from. This is the junction closest to the *previous delivery* in the route (using nearest-neighbour logic).
- ❷ **The ending junction:** which end to exit from. This is the junction closest to the *next gene's* section.
- ❸ **Single-sided or double-sided:** whether to deliver only one side, or both sides in a single sweep (determined by whether the gene appears adjacent to its partner or not).

A powerful **pre-processing optimisation** follows from this structure. Because we know which addresses belong to each section, and which junctions bound each section, we can *pre-compute* the optimal ordering of addresses for all three patterns (one side, both sides, cross-over) at startup. These cached orderings are reused every time the decoder visits that section, avoiding repeated nearest-neighbour calculations during the EA evaluation loop.

11.2 Street Based Routing – Delivery Patterns I

When the decoder processes a gene, it selects one of several **delivery patterns** depending on the start junction, end junction, and whether the section is being delivered as single-sided or double-sided. The figure below illustrates the three main patterns.

SBR Delivery Patterns for a Street Section
Orange circles=junctions, red squares=delivery addresses



11.2 Street Based Routing – Delivery Patterns II

Pattern 1 — One side, same junction (straight walk): The worker enters the section from junction J1 (the nearest junction to the previous delivery), walks along one side of the street delivering to all houses, and exits back through J1. This happens when J1 is also the closest junction to the *next* section.

Pattern 2 — Both sides, U-turn at far end: The worker enters from J1, delivers all houses on side 1 while walking to J2, then crosses the road and delivers all houses on side 2 while walking back to J1. This is the most efficient pattern when both sides are being delivered in one combined sweep.

Pattern 3 — Cross-over (opposite junctions): The worker enters from J1 but must exit at J2 (because J2 is closer to the next section). In this case both sides are delivered in the direction of travel, and the worker exits at J2 having crossed from one side to the other.

11.2 Street Based Routing – Delivery Patterns III

The book shows the actual map-based decoding step-by-step in Fig. 11.2. Below is the crop from the book showing how the first two genes of the example chromosome are decoded onto the real street layout:

11.2 Street Based Routing – Delivery Patterns IV

11.2 Street Based Routing

227



(a) Applying the first pattern based on the first gene *ClunyGdns1*



11.2 Street Based Routing – Decoding Algorithm I

The complete decoding procedure (Algorithm 22) translates a genotype (list of street-section genes) into a **phenotype** (the actual walking route with all individual delivery addresses).

Algorithm 22: SBR Decoding

Input: genotype (ordered list of street sections), problem (start/end depot, all addresses)

Output: Total walking distance of the decoded route

Reading the algorithm step by step:

- Lines 1–6:** Initialise the empty delivery list, set `geneCount` (the current position in the chromosome) to zero, set the previous delivery location to the depot start, and set the accumulated distance to zero.
- Line 7:** Loop through every gene in the chromosome.
- Lines 8–10:** Check whether the current gene equals the next gene. If so, the section is being delivered double-sided (`doubleSided = true`), and we advance the gene counter by one extra step to consume both copies.

11.2 Street Based Routing – Decoding Algorithm II

- Lines 11–13: Record the current and next sections, and remember where the last delivery was made (the tail of the deliveries list so far).
- Line 14: Call **applyPattern** to determine which of the delivery patterns to use for this section, and return the ordered list of addresses for that pattern.
- Lines 15–17: Compute how far the worker walked across this section and add it to the total distance.
- Lines 18–19: Add the section's deliveries to the master list and update `deliveriesLeft` (the remaining count of unvisited addresses).

11.2 Street Based Routing – Decoding Algorithm III

Lines 20–22: After all genes are processed, add the cost of the final walk back to the depot. Crucially, this is weighted by `deliveriesLeft`: the walking distance from the last delivery to the end point, multiplied by the number of addresses still undelivered. This acts as a **penalty** that discourages solutions that leave many deliveries for the end of the route, since the worker would be carrying a heavy load for a long return walk.

Line 22: Return the total distance as the fitness of this genotype.

Key Insight: Pre-cached Orderings

The **applyPattern** subroutine (Algorithm 23) looks up a *pre-computed* ordering of addresses for the required pattern. These orderings are computed once at start-up using the nearest-neighbour heuristic. Because the same section may be visited thousands of times during the EA's evaluation budget, this caching makes decoding extremely fast — much faster than recomputing the optimal ordering each time.

11.2 Street Based Routing – Algorithm 22 Pseudocode I

11.2 Street Based Routing – Algorithm 22 Pseudocode II

Algorithm 22: The SBR Decoding Algorithm (pseudocode)

```
Procedure decoder(genotype, problem)
    deliveries  $\leftarrow$  []
    deliveriesLeft  $\leftarrow$  problem.qtyDeliveries
    geneCount  $\leftarrow$  0
    prevDelivery  $\leftarrow$  problem.start
    dist  $\leftarrow$  0
    while geneCount < genotype.length do
        if genotype[geneCount] = genotype[geneCount + 1] then
            doubleSided  $\leftarrow$  true; geneCount += 1
        current  $\leftarrow$  genotype[geneCount]
        next  $\leftarrow$  genotype[geneCount + 1]
        lastDel  $\leftarrow$  deliveries.tail()
        street  $\leftarrow$  applyPattern(prevDelivery, current, next, doubleSided)
        nextDel  $\leftarrow$  street.head()
        dist += walkingDist(lastDel, nextDel)
        deliveriesLeft -= street.size()
        deliveries  $\leftarrow$  deliveries.append(street)
    lastDel  $\leftarrow$  deliveries.head()
    dist += walkingDist(lastDel, problem.end)  $\times$  deliveriesLeft
    Return dist
EndProcedure
```

11.2 Street Based Routing – Apply Pattern (Algorithm 23) I

Algorithm 23 is called by the decoder for every gene. It decides which delivery pattern to use and returns the ordered list of addresses for that section under the chosen pattern.

11.2 Street Based Routing – Apply Pattern (Algorithm 23) II

Algorithm 23: Apply SBR Pattern (pseudocode)

```
Procedure applyPattern(prevSt, currentSt, nextSt, doubleSided)  
    startJunction  $\leftarrow$  getNearest(currentSt, prevSt)  
    endJunction  $\leftarrow$  getNearest(currentSt, nextSt)  
    deliveries  $\leftarrow$  []  
    if startJunction = endJunction then sameEnd  $\leftarrow$  true  
    if  $\neg$ doubleSided then  
        if currentSt.side1Done = false then  
            deliveries.append(currentSt.side1Dels)  
            currentSt.side1Done  $\leftarrow$  true  
        else deliveries.append(currentSt.side2Dels)  
    if doubleSided and sameEnd then  
        deliveries.append(currentSt.side1Dels)  
        deliveries.append(currentSt.side2Dels)  
    if doubleSided and  $\neg$ sameEnd then  
        deliveries.append(currentSt.crossOverDels)  
    Return deliveries  
EndProcedure
```

11.2 Street Based Routing – Apply Pattern (Algorithm 23) III

Symbol and variable definitions for Algorithm 23:

- `prevSt` — the street section immediately before `currentSt` in the chromosome (or the start depot if this is the first gene).
- `currentSt` — the street section currently being decoded.
- `nextSt` — the next street section in the chromosome (or the end depot if this is the last gene).
- `side1Dels` — the pre-computed, optimally ordered list of addresses on side 1 of `currentSt`.
- `side2Dels` — the pre-computed list for side 2.
- `crossOverDels` — the pre-computed list for the cross-over pattern, where the worker delivers both sides while walking in one direction (entering at J1, exiting at J2).
- `side1Done` — a flag that records whether side 1 has already been delivered on a previous visit; when it is true, the algorithm knows to deliver side 2 instead.

11.2 Street Based Routing – Apply Pattern (Algorithm 23) IV

How to Read This Algorithm

Think of Algorithm 23 as a decision tree with two binary switches: *(1) is the section single-sided or double-sided?* and *(2) does the delivery worker enter and exit from the same junction?* The four combinations yield four different orderings of the addresses, all pre-computed so that this function is simply a look-up.

11.2 Street Based Routing – Genetic Operators I

The SBR EA uses two genetic operators: a **mutation** operator and a **recombination** (crossover) operator. Both operators are designed to respect the structure of the chromosome — in particular, they must not split a double-sided pair of genes in a way that creates an invalid chromosome.

Mutation operator. The mutation operator is relatively simple. It randomly selects one of three equally probable moves:

- 1 **Random relocation:** Select a gene at random and move it to another random position in the chromosome. If the gene is part of a double-sided pair, move the pair together.
- 2 **Junction-based relocation:** Select a gene at random and move it to a position adjacent to another gene that shares a junction with it. This exploits the street topology: genes that share a junction are naturally sequential in good routes.
- 3 **Partner pairing:** If the selected gene is one of a double-sided pair, move it to a position adjacent to its partner. This converts a separate two-visit delivery to a combined one-sweep delivery, which is often shorter.

11.2 Street Based Routing – Genetic Operators II

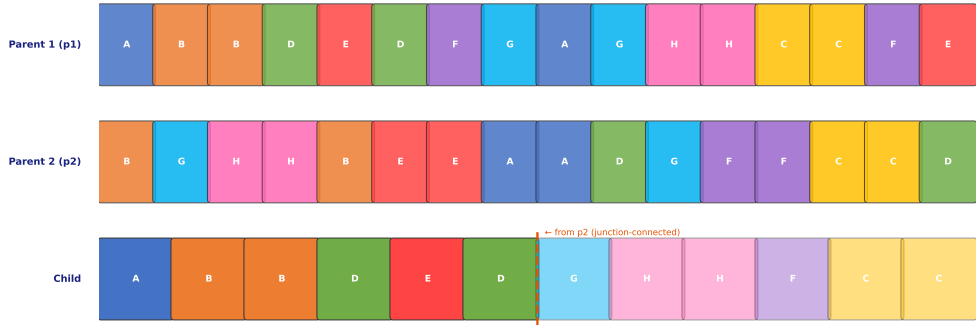
Recombination (crossover) operator. The recombination operator uses a tournament selection of size 2 to choose two parent chromosomes. The child chromosome is then built using a *junction-connected* copying strategy:

- 1 Take the **first gene** from parent 1 (p1).
- 2 For each subsequent position, take the next gene from parent 2 (p2) if the street section it represents is **connected via a junction** to the previous gene already placed in the child.
- 3 If the required gene from p2 is not junction-connected to the previously placed gene, fall back to taking the next gene from p1.
- 4 Continue until the child chromosome is complete.

The key insight is that junction-connected genes form good sub-routes (the delivery worker can walk directly from one section to the next without backtracking), so inheriting connected blocks of genes from a parent tends to produce high-quality children.

11.2 Street Based Routing – Genetic Operators III

SBR Recombination: Child copies connected street sections from both parents



11.2 Street Based Routing – Genetic Operators IV

The crossover figure shows two parent chromosomes (p1 and p2, each 16 genes long) and the resulting child. Blue-tinted genes came from parent 1; darker genes (indicated after the dashed separator) came from parent 2 because they were junction-connected to the previous placed gene.

Worked Example — Crossover Step by Step

Suppose p1 starts with genes [A, B, B, D, E, D, ...] and p2 starts with [B, G, H, H, B, E, ...], where letters represent street sections and each section is labelled by which junctions it connects.

- 1 Take gene A from p1. The child chromosome is [A].
- 2 Is gene B (the first gene of p2) junction-connected to A? Yes — A and B share junction J2. Take B from p2. Child: [A, B].
- 3 Is the next p2 gene junction-connected to B? If yes, continue taking from p2; otherwise fall back to p1.
- 4 Continue until the child has all genes.

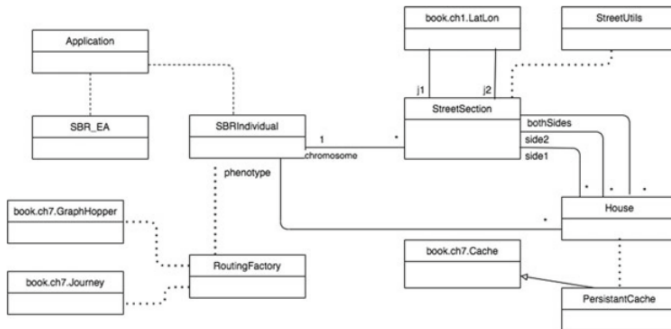
The result is a child that inherits contiguous blocks of street sections that are physically adjacent, producing a route with fewer wasted walking segments between sections.

11.3 Implementation – Overview and Key Classes I

The SBR system is implemented in Java (the book's reference implementation) with the routing distance calculations handled by the GraphHopper routing engine. The architecture consists of several interacting classes.

11.3 Implementation – Overview and Key Classes II

ue id number for each journey, which can then be used as a key to a *HashMap* contains the cached Journey objects.



11.3 The classes that make up the SBR solver

11.3 Implementation – Java Code: Journey Cache

The listing below (Listing 11.1 in the book) shows the Java implementation of the journey distance cache. This is one of the most performance-critical parts of the SBR implementation, because it is called once for every pair of consecutive street sections during every chromosome evaluation.

```
1  public double getJourneyDist(LatLon start, LatLon end, String mode) {
2      // Compute a unique cache key from the hash codes of start and end
3      long key = cantorPair(start.hashCode(), end.hashCode());
4
5      // Try to retrieve a cached result first
6      Double res = cache.get(key);
7      if (res == null) {
8          // Cache miss: call the routing engine to compute the distance
9          options.put(RoutingEngine.OPTION_MODE, mode);
10         res = router.findRoute(start, end, options).getDistanceKM();
11         // Store the result for future reuse
12         cache.put(key, res);
13     }
14     return res;
15 }
16
17 /*
18  * Local cache backed by a HashMap.
19  * key    = Cantor pair of the two endpoint hash codes (unique integer)
20  * value  = walking distance in kilometres
21  */
22 private static HashMap<Long, Double> cache = new HashMap<Long, Double>();
23
```

11.3 Implementation – Setting Up the Network in Code

To define the delivery area, the programmer creates a **StreetSection** object for each section, specifying the street name, the house numbers on each side, and the GPS coordinates of the two bounding junctions. The code listing below (Listing 11.2 in the book) shows how Greenbank Crescent in Edinburgh is added to the SBR chromosome:

```
1 chromo.addStreet(  
2     new StreetSection(  
3         "Greenbank Crescent, Edinburgh",  
4         new int[]{39,41,43,45,47,49,51,53,55,57,59,61,63,65,67,69}, // side 1  
5         new int[]{52,54,56,58,60,62,64,66,68,70,72,74,76,78,80,82}, // side 2  
6         new LatLon(55.9177482, -3.2166029), // junction 1 GPS coordinates  
7         new LatLon(55.915377, -3.216826) // junction 2 GPS coordinates  
8     )  
9 );
```

Reading this code:

- The first argument is the human-readable street name (used for identification and debugging).
- The second and third arguments are arrays of house numbers for side 1 and side 2. Even numbers are on one side, odd numbers on the other (as is conventional in UK addressing).
- The fourth and fifth arguments are the GPS coordinates (latitude, longitude) of the two junctions that bound this section of Greenbank Crescent.

Latitude and Longitude (GPS Coordinates)

A LatLon object stores a pair of decimal numbers: the *latitude* (north–south position, ranging from -90° to 90°)

11.4 Comparison with TSP – The Case Study Area I

To compare SBR-EA against a conventional TSP-EA, the authors use a real-world test case: the **Greenbank area of Edinburgh, Scotland**. This is a residential district with a regular grid-like street layout, typical of urban postal delivery walks.

11.4 Comparison with TSP – The Case Study Area II



11.4 Comparison with TSP – Best Route Found I

The best overall solution found by SBR-EA (6.8 km) is shown below on the real street map of the Greenbank area. The red portion of the route indicates the final walk back to the depot after all deliveries are complete.

11.4 Comparison with TSP – Best Route Found II

11.4 Comparison with TSP

235



11.5 Conclusions I

Street Based Routing is a **representation-and-decoder** combination tailored specifically to urban house-to-house delivery problems. Rather than treating every individual address as an independent decision variable (as TSP does), SBR exploits the natural clustering of addresses into street sections and the connectivity of sections at junctions.

Summary of the SBR Approach

- **Chromosome:** An ordered list of street sections (not individual addresses). Double-sided sections appear twice.
- **Decoder:** Expands each gene into its pre-computed list of delivery addresses using one of three patterns (one-side, U-turn, cross-over) selected based on junction connectivity.
- **Pre-processing:** Optimal address orderings for all patterns are computed once at startup and cached; this makes each chromosome evaluation extremely fast.
- **Operators:** Mutation (random relocation, junction-based relocation, partner pairing) and recombination (junction-connected copying from two parents).

11.5 Conclusions II

Key advantages of SBR over TSP for urban delivery:

- 1 **Massively reduced search space.** For 442 addresses across 48 sections, the SBR space ($48! \approx 10^{61}$) is 10^{917} times smaller than the TSP space ($442!/2 \approx 10^{978}$). The EA can therefore find much better solutions within the same evaluation budget.
- 2 **Built-in domain knowledge.** The SBR representation encodes the principle that adjacent addresses are delivered together. This is always optimal locally, so the EA only needs to optimise global section ordering.
- 3 **Speed.** The combination of a smaller search space and cached address orderings makes SBR-EA approximately $100\times$ faster than TSP-EA on the Greenbank test case.

Limitations and honest caveats:

- SBR *cannot represent* solutions that split a street section's addresses across different parts of the route. A pure TSP could theoretically find such solutions if they happened to be shorter. The authors acknowledge this, but argue that in practice the speed advantage of SBR outweighs the possibility of missing such exotic solutions.

11.5 Conclusions III

- The comparison in this chapter is intentionally limited in scope (one study area, one budget, 10 runs). Larger studies could give more robust statistical conclusions.
- The implementation uses a routing engine (GraphHopper) to compute walking distances. Any system that cannot call a routing API would need to fall back to Euclidean or Manhattan distances.

11.5 Conclusions IV

The Big Takeaway

Domain knowledge, when correctly encoded into the chromosome representation and decoding process, can reduce the effective search space by many orders of magnitude and produce dramatically better solutions within the same computational budget. SBR is a prime example of this principle: by exploiting the structure of urban streets, it finds routes that are both shorter and found faster than a general-purpose TSP approach.

Broader applicability. The SBR principle generalises to any problem where deliveries or visits cluster naturally into groups: political canvassing, census data collection, utility meter reading, and any other house-to-house operation in urban areas. The key requirement is that the groups (street sections) be well-defined, bounded by identifiable transition points (junctions), and that within-group ordering can be pre-optimised.

11.5 Conclusions V

References

- Urquhart, N., B. Paechter, and K. Chisholm. 2001. Street-Based Routing Using an Evolutionary Algorithm. In *Applications of Evolutionary Computing*, ed. E.J.W. Boers, 495–504. Springer.
- Urquhart, Neil. 2002. *Solving a Real-World Routing Problem using Evolutionary Algorithms and Agents*, PhD Thesis, Edinburgh Napier University.
- Pigeon, Steven. n.d. Pairing Function. *From MathWorld – A Wolfram Web Resource*.
<https://mathworld.wolfram.com/PairingFunction.html>

Appendix A – Worked Example: Full SBR Decoding I

This appendix walks through a complete, concrete decoding of a small SBR chromosome to show every step of Algorithm 22.

Setup. Suppose our delivery area has three street sections:

Section	Addresses side 1	Addresses side 2
AlphaRd1	1, 3, 5, 7, 9	2, 4, 6, 8, 10
BetaLane1	11, 13, 15	12, 14, 16
GammaAv1	17, 19, 21	— (one side only)

(Alpha and Beta are double-sided; Gamma is one-sided.) The depot is at junction J0. Walking distances between consecutive houses within a section are 30 m each; distances between junctions and neighbouring houses are 20 m; distance from the depot to the first house on each section is 100 m.

Chromosome to decode:

[GammaAv1, AlphaRd1, AlphaRd1, BetaLane1, BetaLane1]

Note: AlphaRd1 and BetaLane1 each appear twice (both sides).

Appendix A – Worked Example: Full SBR Decoding II

Step-by-step decoding:

Gene 1 — GammaAv1 (single-sided):

- Previous location: depot (J0).
- Start junction: the junction of GammaAv1 nearest to J0. Suppose this is junction J1.
- One-side pattern: deliver addresses 17, 19, 21 in order from J1 to J2. Walking distance within section: $3 \times 30 = 90$ m.
- Distance from depot to first address (#17): 100 m.
- Subtotal so far: $100 + 90 = 190$ m.

Genes 2 and 3 — AlphaRd1 (double-sided, adjacent):

- Gene 2 equals Gene 3, so `doubleSided = true`.
- U-turn pattern: deliver side 1 (1, 3, 5, 7, 9) walking J1 to J3; then deliver side 2 (10, 8, 6, 4, 2) walking back J3 to J1.
- Walking distance side 1: $4 \times 30 = 120$ m. U-turn: 0 m extra. Walking distance side 2: $4 \times 30 = 120$ m.

Appendix A – Worked Example: Full SBR Decoding III

- Walking from end of Gamma to start of Alpha: suppose 80 m (between J2 and J1).
- Subtotal this section: $80 + 120 + 120 = 320$ m.
- Cumulative total: $190 + 320 = 510$ m.

Genes 4 and 5 — BetaLane1 (double-sided, adjacent):

- Same logic: U-turn pattern; 3 houses per side.
- Walking side 1: $2 \times 30 = 60$ m; side 2: $2 \times 30 = 60$ m.
- Walking from Alpha to Beta: suppose 90 m.
- Subtotal: $90 + 60 + 60 = 210$ m.
- Cumulative total: $510 + 210 = 720$ m.

Appendix A – Worked Example: Full SBR Decoding IV

Return walk to depot: Suppose 150 m from the last address on Beta back to the depot. At this point $\text{deliveriesLeft} = 0$ (all addresses delivered), so the penalty term is $150 \times 0 = 0$ m extra.

Total route distance: $720 + 150 = 870$ m.

Note on the Penalty Term

The penalty term $\text{walkingDist}(\text{lastDel}, \text{end}) \times \text{deliveriesLeft}$ is zero when all deliveries are made before the return walk (as here). It becomes large when deliveries are spread unevenly, forcing a long return trip while still carrying packages. This steers the EA towards solutions that front-load deliveries.

Appendix B – Worked Example: Cantor Pairing in the Cache I

This example traces the cache behaviour during a small EA evaluation run to illustrate how Cantor pairing prevents redundant routing calls.

Setup. Suppose the routing engine takes 200 ms to compute each walking distance. Our EA evaluates 10,000 chromosomes. Each chromosome has on average 30 gene-to-gene transitions. Without caching, total routing calls = $10,000 \times 30 = 300,000$, taking $300,000 \times 0.2 \text{ s} = 60,000 \text{ s} \approx 17 \text{ hours}$.

With caching. For a delivery area with 48 street sections, there are at most $48 \times 48 = 2,304$ unique ordered pairs (section A followed by section B). After the cache is “warm” (all pairs seen at least once), every subsequent routing query is a HashMap look-up taking $\approx 0.001 \text{ ms}$.

Warm-up phase: at most 2,304 routing calls, costing $2,304 \times 0.2 = 461 \text{ s} \approx 8 \text{ minutes}$.

Steady-state phase: all 10,000 chromosomes evaluated with only HashMap look-ups, costing essentially zero routing engine time.

Appendix B – Worked Example: Cantor Pairing in the Cache II

Cantor Key Computation for a Specific Pair

Section Greenbank Crescent N has hash code $x = 12345$. Section Greenbank Drive E has hash code $y = 67890$.

$$\begin{aligned} z &= \frac{1}{2}(12345 + 67890)(12345 + 67890 + 1) + 67890 \\ &= \frac{1}{2} \times 80235 \times 80236 + 67890 \\ &= \frac{1}{2} \times 6,437,823,060 + 67890 \\ &= 3,218,911,530 + 67890 \\ &= 3,218,979,420 \end{aligned}$$

The walking distance from GreenBank Crescent N to Greenbank Drive E is stored in the HashMap under the key 3,218,979,420. This is a 32-bit integer that fits comfortably in a Java long.

Appendix C – Solution Space Size: Numbers in Perspective I

Understanding just how large the TSP solution space is helps motivate the need for SBR. The following table puts the numbers in context.

Quantity	Approximate value	Notes
Atoms in observable universe	10^{80}	
TSP solutions for 442 addresses	5.5×10^{978}	$= 442!/2$
SBR solutions for 48 sections	1.2×10^{61}	$= 48!$
Ratio (TSP / SBR)	$\approx 10^{917}$	SBR space is this many times smaller
TSP solutions for 20 cities	6.1×10^{16}	Still 61 quadrillion routes
TSP solutions for 10 cities	1.8×10^5	181,440 — searchable by brute force

A physical analogy. Imagine you have a deck of 442 playing cards (with additional cards added beyond the standard 52). The number of ways to shuffle those cards is $442!/2 \approx 5.5 \times 10^{978}$. If you could shuffle the deck once per nanosecond and had been doing so since the Big Bang (≈ 13.8 billion years ago), you would have explored approximately 4.4×10^{26} arrangements — a number so small compared to 10^{978} that you would not even have made a dent in the full space.

Appendix C – Solution Space Size: Numbers in Perspective II

This illustrates why evolutionary algorithms with smart representations are not merely a convenience: they are a necessity for practical combinatorial optimisation at realistic problem sizes.